

Credit Card Approval Prediction

Project Description:

The **Credit Card Approval Prediction System** is a Machine Learning-based web application developed to automate the process of evaluating credit card applications. Financial institutions receive thousands of applications every day, making manual verification time-consuming and prone to human error. This project addresses these challenges by using machine learning algorithms to analyze applicant information and predict whether a credit card application should be approved or rejected.

The application is built using the **Flask** framework and integrates an **XGBoost Classifier** for prediction. The system accepts various applicant details such as age, annual income, employment duration, education level, occupation, housing type, credit history, loan amount, existing loan balance, number of credit inquiries, monthly income, and employment status. These features are preprocessed using **One-Hot Encoding** for categorical attributes and **Standard Scaling** for numerical attributes before being passed to the trained machine learning model.

The project compares multiple machine learning algorithms, including **Logistic Regression**, **Decision Tree**, **Random Forest**, and **XGBoost**, and automatically selects the model with the highest validation accuracy for deployment. The prediction results include the approval status, confidence score, and a meaningful explanation for the decision. Additionally, the application maintains a prediction history during the user session, providing a better user experience.

The objective of this project is to demonstrate how artificial intelligence and machine learning can improve the speed, consistency, and reliability of credit card approval decisions while reducing manual effort and operational costs.

Scenarios:

Scenario 1: Applicant with Good Financial Profile

A customer applies for a credit card by providing all the required information through the web application. The applicant has a good credit history, stable employment, high annual income, low existing loan balance, and very few credit inquiries.

The system preprocesses the input data and passes it to the trained XGBoost model. The model predicts that the application is approved with a high confidence score. The result page displays the approval status along with an explanation stating that the applicant has a favorable debt-to-income ratio and excellent credit history.

Scenario 2: Applicant with High Financial Risk

Another applicant submits an application with a poor credit history, high existing loan balance, multiple recent credit inquiries, and unstable employment.

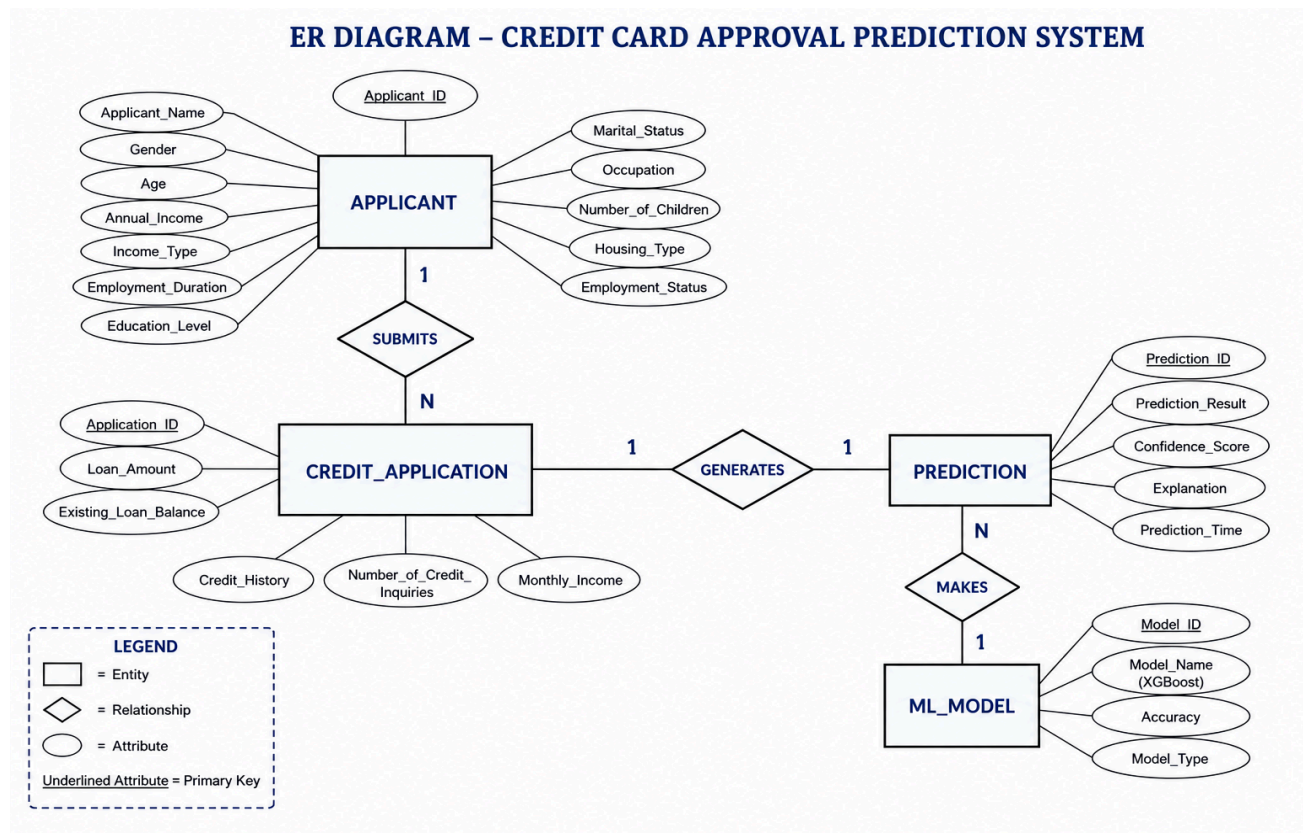
After preprocessing the input features, the machine learning model predicts that the application should be rejected. The application displays the rejection status together with an explanation indicating that the applicant presents a high financial risk due to poor credit history and excessive debt.

Scenario 3: Borderline Applicant

An applicant has an average annual income and moderate employment history but has several existing loans and a relatively high debt-to-income ratio.

The model evaluates all the input features collectively and predicts the final decision based on learned patterns. Depending on the applicant's financial profile, the application displays either an approval or rejection result along with the prediction confidence and explanation.

Technical Architecture:



Description

The Credit Card Approval Prediction System follows a **three-tier architecture** consisting of the Presentation Layer, Application Layer, and Machine Learning Layer.

Presentation Layer

The Presentation Layer is developed using **HTML, CSS, and JavaScript**. It provides a responsive and user-friendly interface where users can enter applicant information, submit the application, and view prediction results. The interface also includes pages for prediction history and application status.

Application Layer

The Application Layer is implemented using the **Flask Framework**. It manages user requests, validates input data, performs preprocessing, communicates with the machine learning model, and generates prediction results. Flask routing handles navigation between the Home, Prediction, Result, and History pages.

Machine Learning Layer

The Machine Learning Layer is responsible for data preprocessing and prediction. Numerical features are standardized using **StandardScaler**, while categorical features are converted into numerical representations using **OneHotEncoder**. The processed data is then passed to the trained **XGBoost Classifier**, which predicts whether the credit card application should be approved or rejected.

The trained model, encoder, and scaler are serialized using **Joblib**, allowing the Flask application to load them efficiently during runtime.

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Machine Learning Fundamentals:** Scikit-learn Documentation
- **XGBoost Knowledge:** XGBoost Documentation
- **Python Programming Proficiency:** Python Documentation
- **Data Analysis with Pandas & NumPy:** Pandas and NumPy Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Model Serialization using Joblib:** Joblib Documentation
- **Development Environment Setup:** Flask Installation Guide
- **Version Control with Git:** Git Documentation
- **Virtual Environment Setup:** Python venv Documentation
- **Web Browser for Testing:** Google Chrome / Microsoft Edge
- **Code Editor:** Visual Studio Code or PyCharm Documentation

Project Working Flow

Step 1: Import Required Libraries

The first step in developing the Credit Card Approval Prediction System is importing all the required Python libraries. These libraries provide functionalities for data manipulation, preprocessing, machine learning model training, evaluation, hyperparameter tuning, model serialization, and file management.

The **NumPy** library is used for performing numerical computations and generating synthetic data. **Pandas** is used for creating and managing datasets in tabular format. **Scikit-learn** provides utilities for preprocessing, train-test splitting, model evaluation, and implementing machine learning algorithms such as Logistic Regression, Decision Tree, and Random Forest. **XGBoost** is used for building a high-performance classification model. Finally, **Joblib** is used to save the trained machine learning model, encoder, and scaler for later use in the Flask web application.

Code

```
import os
import joblib
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
from xgboost import XGBClassifier
```

Step 2: Generate the Synthetic Credit Card Dataset

Since a real banking dataset is not used, the project generates a synthetic dataset using the `generate_synthetic_data()` function. This function creates realistic applicant records with different personal, employment, and financial attributes.

The generated dataset contains approximately **1500 applicant records**. Each applicant includes information such as gender, age, annual income, income type, employment duration, education level, marital status, occupation, housing type, credit history, loan amount, existing loan balance, monthly income, employment status, and approval status.

Random values are generated using NumPy while maintaining realistic relationships between different financial attributes.

Code

```
def generate_synthetic_data(num_samples=1500):  
    np.random.seed(42)  
  
    genders = ['Male', 'Female']  
    income_types = [  
        'working',  
        'Commercial associate',  
        'Pensioner',  
        'State servant',  
        'Student'  
    ]  
  
    education_levels = [  
        'Higher education',  
        'Secondary / secondary special',  
        'Incomplete higher',  
        'Lower secondary',  
        'Academic degree'  
    ]  
  
    marital_statuses = [  
        'Married',  
        'Single / not married',  
        'Civil marriage',  
        'Separated',  
        'Widow'  
    ]  
  
    occupations = [  
        'Laborers',  
        'Core staff',  
        'Sales staff',  
        'Managers',  
        'Drivers',  
        'High skill tech staff',  
        'Accountants',  
        'Medicine staff',  
        'Security staff',  
        'Cooking staff',  
        'Cleaning staff',  
        'Private service staff',  
        'Low-skill Laborers',  
        'Waiters/barmen staff',  
        'Secretaries',  
        'HR staff',  
        'IT staff',  
        'Realty agents'  
    ]
```

The function also generates numerical attributes such as annual income, loan amount, monthly income, employment duration, and existing loan balance using different probability distributions to produce realistic applicant information.

Step 3: Calculate Applicant Approval Status

After generating all applicant details, the project calculates a credit approval score for every applicant.

The approval score depends on several financial factors including:

- Credit History
- Debt-to-Income Ratio
- Loan-to-Income Ratio
- Number of Credit Inquiries
- Employment Status
- Education Level

Each factor either increases or decreases the applicant's overall score. Finally, a small amount of random noise is added to make the dataset more realistic.

Applicants whose final score is **65 or above** are marked as **Approved**, while the remaining applicants are marked as **Rejected**.

Code

```
approved = []

for idx in range(num_samples):

    score = 60

    if credit_history[idx] == 'Good':
        score += 35
    else:
        score -= 45

    debt_to_income = existing_loan_balance[idx] / annual_income[idx]

    if debt_to_income > 1.2:
        score -= 25
    elif debt_to_income > 0.5:
        score -= 10

    loan_to_income = loan_amount[idx] / annual_income[idx]

    if loan_to_income > 3.0:
        score -= 20
    elif loan_to_income > 1.5:
        score -= 10

    inq = credit_inquiries[idx]

    if inq > 4:
        score -= 25
    elif inq > 2:
        score -= 10

    if employment_status[idx] == 'Unemployed':
        score -= 20

    noise = np.random.normal(0, 8)

    final_score = score + noise

    approved.append(1 if final_score >= 65 else 0)
```

This scoring mechanism simulates the decision-making process commonly used by financial institutions while creating a balanced dataset for machine learning.

Step 4: Create and Save the Dataset

After calculating the approval status for all applicants, the generated information is stored inside a Pandas DataFrame.

The DataFrame contains all applicant features along with the target column **Approved**. The dataset is then saved as **credit_card_data.csv** inside the **dataset** folder so that it can be used during model training.

Code

```
df = pd.DataFrame({
    'Gender': gender,
    'Age': np.round(age, 1),
    'Annual Income': np.round(annual_income, 2),
    'Income Type': income_type,
    'Employment Duration': np.round(employment_duration, 1),
    'Education Level': education_level,
    'Marital Status': marital_status,
    'Occupation': occupation,
    'Number of Children': num_children,
    'Housing Type': housing_type,
    'Credit History': credit_history,
    'Loan Amount': np.round(loan_amount, 2),
    'Existing Loan Balance': np.round(existing_loan_balance, 2),
    'Number of Credit Inquiries': credit_inquiries,
    'Monthly Income': np.round(monthly_income, 2),
    'Employment Status': employment_status,
    'Approved': approved
})

os.makedirs('dataset', exist_ok=True)
dataset_path = os.path.join('dataset', 'credit_card_data.csv')
df.to_csv(dataset_path, index=False)
```

The generated dataset is now ready for preprocessing, machine learning model training, and evaluation. This marks the completion of the initial data generation phase of the Credit Card Approval Prediction System.

Step 5: Define Feature Columns and Split the Dataset

After generating the dataset, the next step is to separate the input features from the target variable. The dataset contains both categorical and numerical features, which are stored separately for preprocessing. The target variable, **Approved**, represents whether a credit card application is approved or rejected.

The dataset is then divided into training and testing sets using the **train_test_split()** function. Eighty percent of the data is used for training the machine learning models, while the remaining twenty percent is reserved for evaluating their performance.

Code

```
categorical_cols = [  
    'Gender', 'Income Type', 'Education Level',  
    'Marital Status', 'Occupation',  
    'Housing Type', 'Credit History',  
    'Employment Status'  
]  
  
numerical_cols = [  
    'Age', 'Annual Income',  
    'Employment Duration',  
    'Number of Children',  
    'Loan Amount',  
    'Existing Loan Balance',  
    'Number of Credit Inquiries',  
    'Monthly Income'  
]  
  
X = df.drop(columns=['Approved'])  
y = df['Approved']  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

Separating the dataset into training and testing data allows the machine learning models to be evaluated on unseen records, ensuring reliable performance measurement.

Step 6: Preprocess the Training and Testing Data

Machine learning algorithms require numerical input values. Therefore, categorical attributes such as Gender, Occupation, Credit History, and Housing Type are converted into numerical values using **One-Hot Encoding**. Numerical attributes such as Annual Income, Loan Amount, and Monthly Income are standardized using **StandardScaler**.

After preprocessing, the encoded categorical features and scaled numerical features are combined into a single feature matrix for both training and testing datasets.

Code

```
print("Fitting Encoder and Scaler...")

encoder = OneHotEncoder(
    sparse_output=False,
    handle_unknown='ignore'
)

scaler = StandardScaler()

X_train_cat = encoder.fit_transform(
    X_train[categorical_cols]
)

X_train_num = scaler.fit_transform(
    X_train[numerical_cols]
)

X_test_cat = encoder.transform(
    X_test[categorical_cols]
)

X_test_num = scaler.transform(
    X_test[numerical_cols]
)

X_train_processed = np.hstack(
    (X_train_num, X_train_cat)
)

X_test_processed = np.hstack(
    (X_test_num, X_test_cat)
)
```

This preprocessing step ensures that all input features are transformed into a suitable numerical format before model training begins.

Step 7: Train Baseline Machine Learning Models

To identify the most suitable algorithm for credit card approval prediction, the project initially trains three baseline machine learning models.

The selected models are:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier

Each model is trained using the processed training dataset and later evaluated using the testing dataset.

Code

```
models = {  
    'Logistic Regression':  
        LogisticRegression(  
            random_state=42,  
            max_iter=500  
        ),  
  
    'Decision Tree':  
        DecisionTreeClassifier(  
            random_state=42  
        ),  
  
    'Random Forest':  
        RandomForestClassifier(  
            n_estimators=100,  
            max_depth=12,  
            random_state=42  
        )  
}
```

These baseline models provide a performance benchmark before applying the advanced XGBoost algorithm.

Step 8: Evaluate the Baseline Models

Once all baseline models are trained, their prediction accuracy is calculated using the testing dataset. Along with the accuracy score, a detailed classification report is generated for each model to evaluate precision, recall, and F1-score.

The evaluation results are stored for comparison with the XGBoost model in later stages.

Code

```
print("Training baseline models...")

results = []

for model_name, model in models.items():

    model.fit(
        X_train_processed,
        y_train
    )

    y_pred = model.predict(
        X_test_processed
    )

    acc = accuracy_score(
        y_test,
        y_pred
    )

    results.append(
        (model_name, model, acc)
    )

    print(
        f"{model_name} Validation Accuracy: {acc * 100:.2f}%"
    )

    print(
        classification_report(
            y_test,
            y_pred
        )
    )
```

The validation accuracy and classification report help compare the performance of the Logistic Regression, Decision Tree, and Random Forest models. Based on these results, the project proceeds to optimize the XGBoost classifier using hyperparameter tuning, which is covered in the next step.

Step 9: Hyperparameter Tuning of the XGBoost Model

After evaluating the baseline machine learning models, the next step is to improve the performance of the **XGBoost Classifier**. Instead of using default parameters, the project performs **hyperparameter tuning** using **RandomizedSearchCV**. This process searches through different combinations of parameter values to identify the configuration that produces the highest prediction accuracy.

The tuning process optimizes important parameters such as the number of trees, learning rate, maximum tree depth, subsampling ratio, column sampling ratio, and minimum child weight. A five-fold cross-validation technique is used to ensure that the selected parameters generalize well to unseen data.

Code

```
print("Training XGBoost with hyperparameter tuning...")

xgb = XGBClassifier(
    random_state=42,
    eval_metric='logloss'
)

param_dist = {
    'n_estimators': [100, 150, 200],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.05, 0.1],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0],
    'min_child_weight': [1, 3, 5]
}
```

The parameter combinations are then evaluated using **RandomizedSearchCV** to obtain the best-performing XGBoost model.

Code

```
random_search = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_dist,
    n_iter=10,
    scoring='accuracy',
    cv=5,
    verbose=1,
    random_state=42,
    n_jobs=-1
)

random_search.fit(
    X_train_processed,
    y_train
)
```

Hyperparameter tuning significantly improves the prediction capability of the XGBoost model by selecting the most appropriate parameter combination.

Step 10: Evaluate the Optimized XGBoost Model

After the tuning process is completed, the optimized XGBoost model is selected as the final model. The model is then tested using the testing dataset to measure its prediction performance.

The model predicts the approval status for every applicant in the testing dataset, and the overall accuracy along with the classification report is generated.

Code

```
best_xgb = random_search.best_estimator_  
  
y_pred_xgb = best_xgb.predict(  
    X_test_processed  
)  
  
xgb_accuracy = accuracy_score(  
    y_test,  
    y_pred_xgb  
)  
  
print(f"XGBoost Validation Accuracy: {xgb_accuracy * 100:.2f}%")  
  
print(  
    classification_report(  
        y_test,  
        y_pred_xgb  
    )  
)
```

The optimized XGBoost model achieves better prediction accuracy than the previously trained baseline models. Therefore, it is selected as the final machine learning model for the Credit Card Approval Prediction System.

Step 11: Compare All Machine Learning Models

Once every machine learning algorithm has been evaluated, the project compares their performance based on validation accuracy. This comparison helps determine which model provides the most reliable predictions.

The algorithms compared in the project include:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- XGBoost Classifier

The model with the highest validation accuracy is selected for deployment in the Flask application.

Code

```
results.append(  
    (  
        "XGBoost",  
        best_xgb,  
        xgb_accuracy  
    )  
)  
  
results.sort(  
    key=lambda x: x[2],  
    reverse=True  
)  
  
best_model_name, best_model, best_accuracy = results[0]  
  
print(f"Best Model: {best_model_name}")  
print(f"Accuracy: {best_accuracy * 100:.2f}%")
```

The comparison confirms that **XGBoost** provides superior prediction performance due to its ensemble learning capability, efficient gradient boosting mechanism, and robustness in handling structured tabular data.

Step 12: Save the Trained Machine Learning Model

After selecting the best-performing model, the project saves the trained XGBoost model along with the fitted **StandardScaler** and **OneHotEncoder**. Saving these files prevents the need to retrain the model every time the web application is executed.

The saved files are later loaded directly by the Flask application during startup.

Code

```
os.makedirs("model", exist_ok=True)

joblib.dump(
    best_model,
    "model/model.pkl"
)

joblib.dump(
    scaler,
    "model/scaler.pkl"
)

joblib.dump(
    encoder,
    "model/encoder.pkl"
)

print("Model, scaler, and encoder saved successfully.")
```

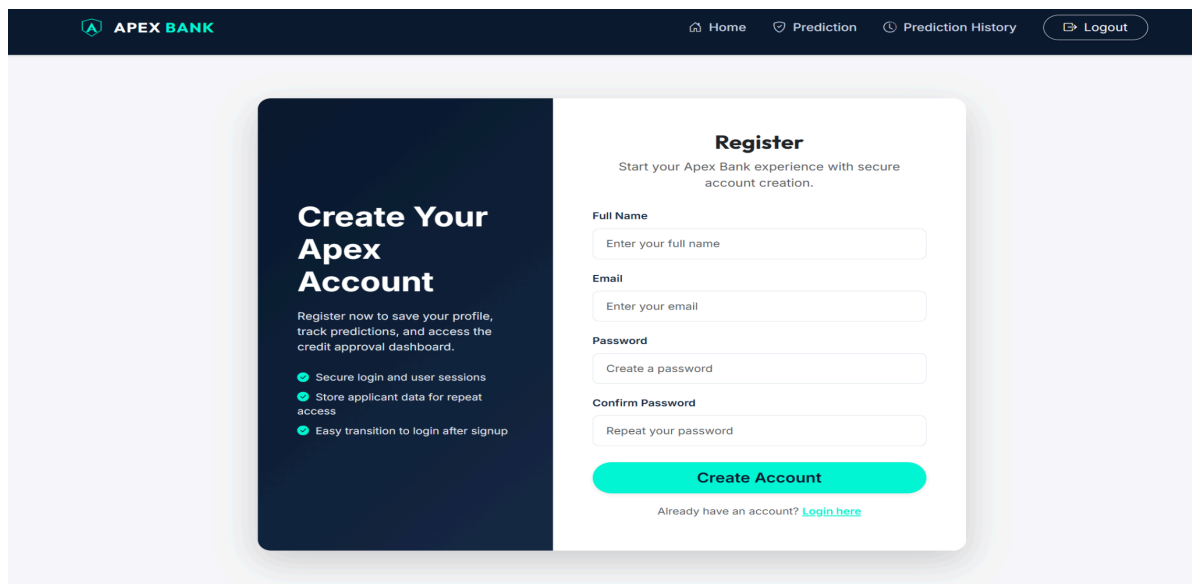
The generated files include:

- **model.pkl** – Stores the trained XGBoost classification model.
- **scaler.pkl** – Stores the fitted StandardScaler used for numerical features.
- **encoder.pkl** – Stores the fitted One-Hot Encoder used for categorical features.

These files form the foundation of the prediction module used by the Flask web application. Instead of retraining the model, the application simply loads these files and performs real-time credit card approval prediction based on the applicant's input

Website Working Flow:

Step 1: User Registration



The screenshot shows the 'Register' page of the Apex Bank web application. The page has a dark blue header with the 'APEX BANK' logo and navigation links for Home, Prediction, Prediction History, and Logout. The main content area is divided into two sections. On the left, a dark blue box contains the text 'Create Your Apex Account' and a list of benefits: 'Secure login and user sessions', 'Store applicant data for repeat access', and 'Easy transition to login after signup'. On the right, a white box contains the 'Register' form. The form includes fields for 'Full Name', 'Email', 'Password', and 'Confirm Password', each with a placeholder text. A green 'Create Account' button is at the bottom of the form, and a link to 'Login here' is provided for existing users.

Register

Start your Apex Bank experience with secure account creation.

Full Name

Enter your full name

Email

Enter your email

Password

Create a password

Confirm Password

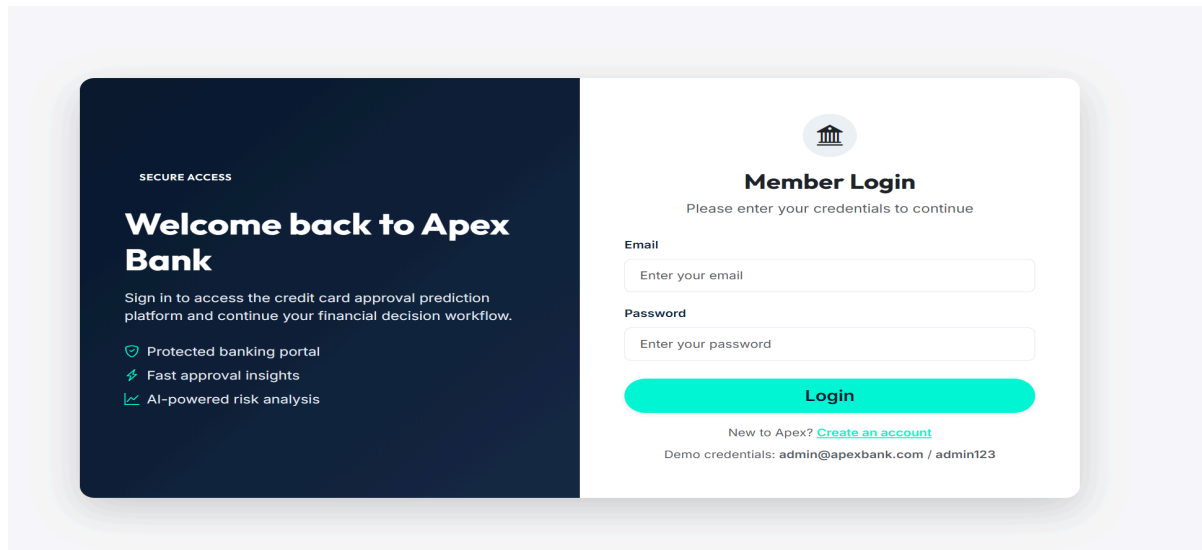
Repeat your password

Create Account

Already have an account? [Login here](#)

The first step of the application is user registration. New users create an account by entering their Full Name, Email Address, Password, and Confirm Password. The system validates the entered information and securely stores the user's credentials in the database. After successful registration, the user can log in to access all features of the Credit Card Approval Prediction System.

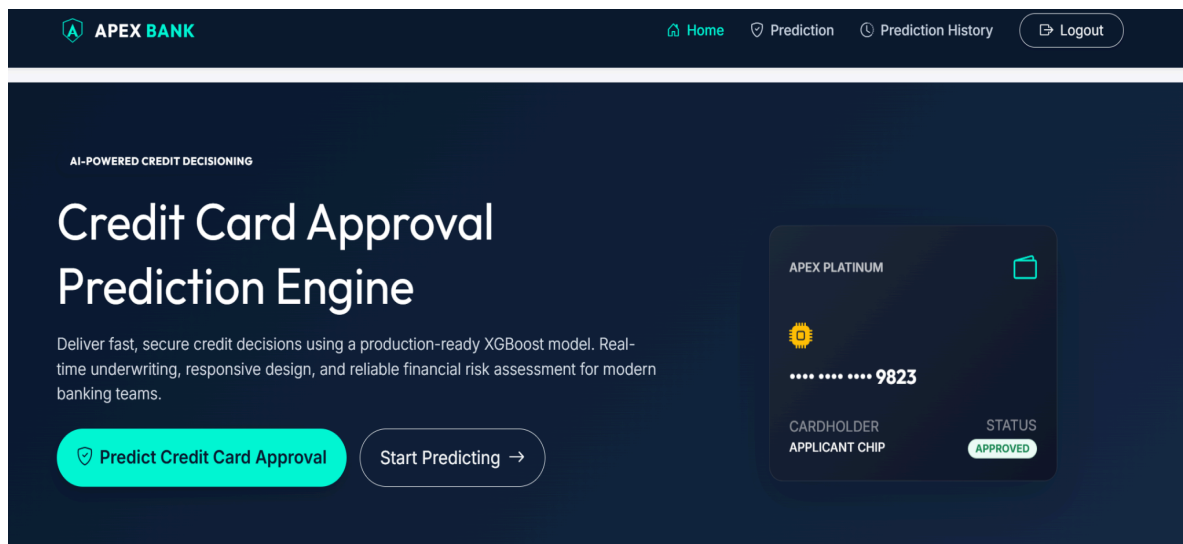
Step 2: User Login



The login form is titled "Member Login" and includes a sub-header "Please enter your credentials to continue". It features two input fields: "Email" with the placeholder "Enter your email" and "Password" with the placeholder "Enter your password". A prominent red "Login" button is positioned below the password field. To the right of the button, there is a link for "New to Apex? Create an account" and demo credentials: "Demo credentials: admin@apexbank.com / admin123". On the left side of the form, a dark blue sidebar contains the text "Welcome back to Apex Bank" and "Sign in to access the credit card approval prediction platform and continue your financial decision workflow." Below this, three bullet points highlight features: "Protected banking portal", "Fast approval insights", and "AI-powered risk analysis".

After creating an account, users log in using their registered email address and password. The Flask backend authenticates the credentials stored in the database. Once authentication is successful, a secure user session is created, and the user is redirected to the application's home page.

Step 3: Home Dashboard



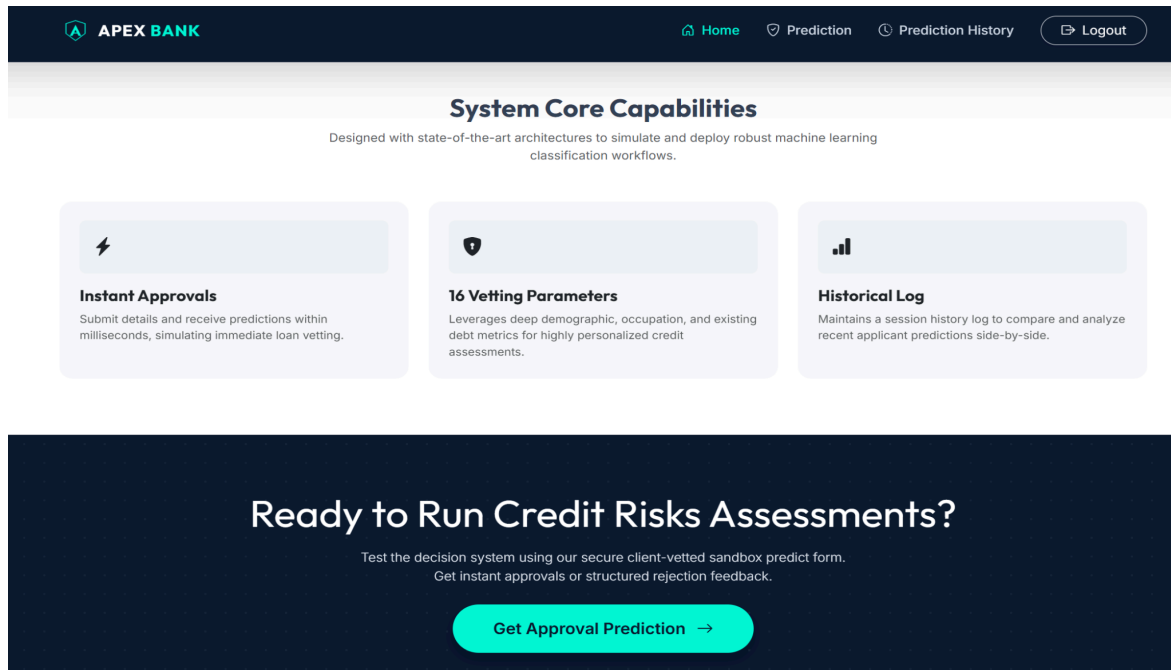
The home dashboard features a dark blue header with the "APEX BANK" logo and navigation links: "Home", "Prediction", "Prediction History", and "Logout". The main content area is titled "AI-POWERED CREDIT DECISIONING" and "Credit Card Approval Prediction Engine". It describes the system as a "production-ready XGBoost model" for "real-time underwriting, responsive design, and reliable financial risk assessment". Two buttons are present: a red "Predict Credit Card Approval" button and a white "Start Predicting" button with a right arrow. On the right, a card displays "APEX PLATINUM" with a card number "9823", the label "CARDHOLDER APPLICANT CHIP", and a "STATUS APPROVED" badge.

System Core Capabilities

Designed with state-of-the-art architectures to simulate and deploy robust machine learning classification workflows.

After login, the user is welcomed to the Home Dashboard. This page introduces the Credit Card Approval Prediction System and explains its purpose. It highlights that the system uses a trained XGBoost Machine Learning model to evaluate applicant information and predict whether a credit card application is likely to be approved or rejected. Navigation links are also provided for Prediction, Prediction History, and Logout.

Step 4: System Core Capabilities



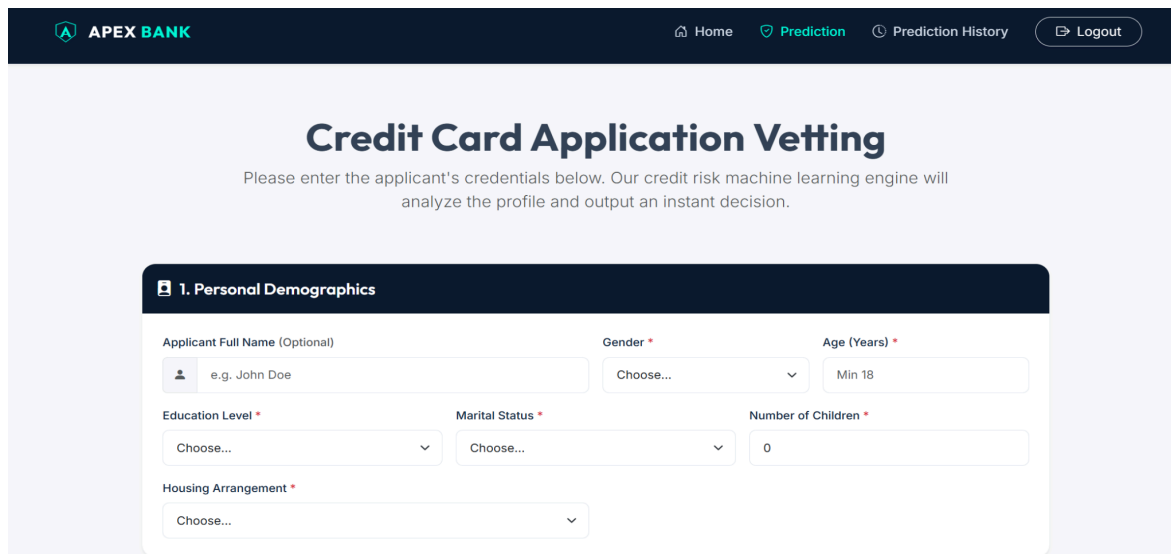
The screenshot shows the APEX BANK Home Dashboard. The header includes the APEX BANK logo and navigation links: Home, Prediction, Prediction History, and Logout. The main section is titled "System Core Capabilities" and features three cards:

- Instant Approvals**: Submit details and receive predictions within milliseconds, simulating immediate loan vetting.
- 16 Vetting Parameters**: Leverages deep demographic, occupation, and existing debt metrics for highly personalized credit assessments.
- Historical Log**: Maintains a session history log to compare and analyze recent applicant predictions side-by-side.

Below the cards is a dark blue banner with the text "Ready to Run Credit Risks Assessments?" and a button "Get Approval Prediction →".

The next section of the home page describes the major features of the system. It highlights Instant Approval Prediction, Evaluation using 16 Applicant Parameters, and Prediction History Management. This section helps users understand how the application performs fast and reliable credit risk assessment using machine learning.

Step 5: Credit Card Application Form – Personal Demographics

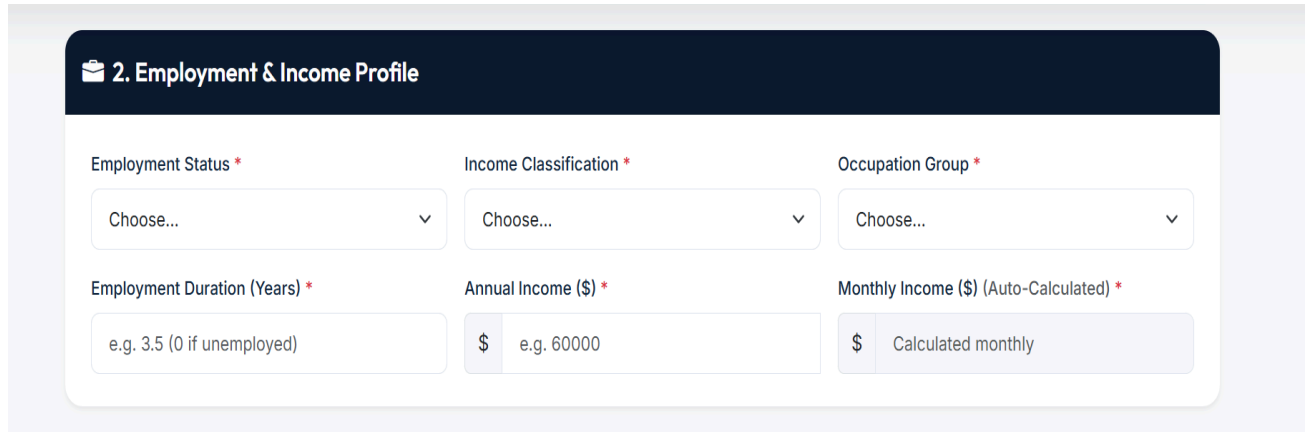


The screenshot shows the "Credit Card Application Vetting" form. The header includes the APEX BANK logo and navigation links: Home, Prediction, Prediction History, and Logout. The main section is titled "Credit Card Application Vetting" and features a form with the following fields:

- Applicant Full Name (Optional)**: Text input field with placeholder "e.g. John Doe".
- Gender ***: Dropdown menu with "Choose..." option.
- Age (Years) ***: Text input field with placeholder "Min 18".
- Education Level ***: Dropdown menu with "Choose..." option.
- Marital Status ***: Dropdown menu with "Choose..." option.
- Number of Children ***: Text input field with placeholder "0".
- Housing Arrangement ***: Dropdown menu with "Choose..." option.

When the user clicks on the Prediction menu, the Credit Card Application Form opens. The first section collects the applicant's personal information, including Full Name, Gender, Age, Education Level, Marital Status, Number of Children, and Housing Arrangement. These demographic details are essential input features for the machine learning model.

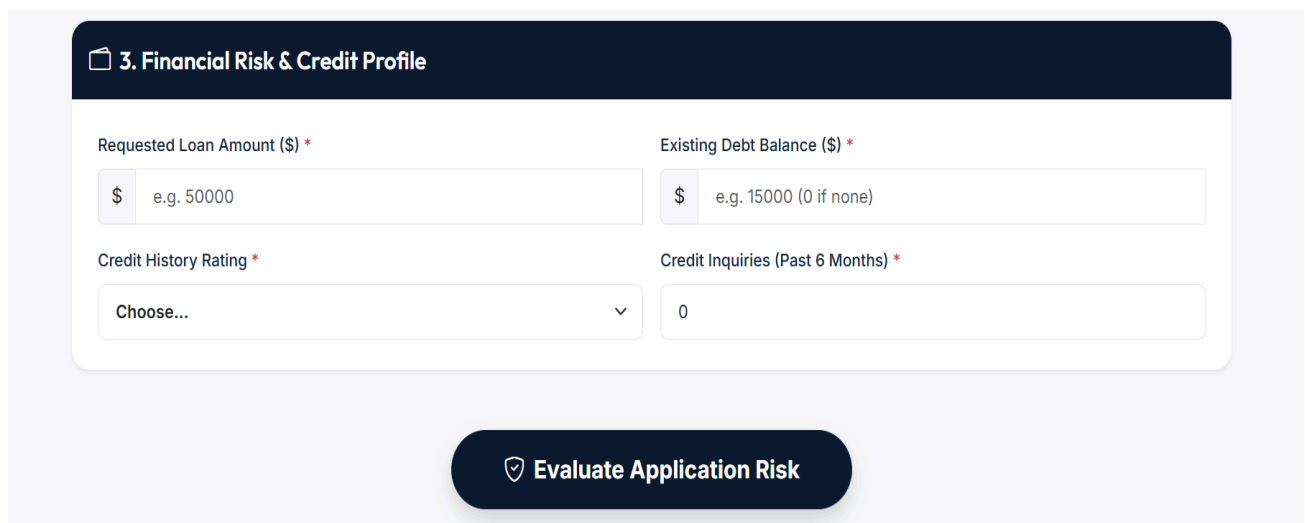
Step 6: Employment and Income Details



The screenshot shows a form titled "2. Employment & Income Profile". It contains six input fields arranged in two rows. The first row has three dropdown menus: "Employment Status *", "Income Classification *", and "Occupation Group *", each with a "Choose..." placeholder and a downward arrow. The second row has three input fields: "Employment Duration (Years) *" with a placeholder "e.g. 3.5 (0 if unemployed)", "Annual Income (\$) *" with a dollar sign icon and a placeholder "e.g. 60000", and "Monthly Income (\$) (Auto-Calculated) *" with a dollar sign icon and a placeholder "Calculated monthly".

The second section collects employment-related information such as Employment Status, Income Classification, Occupation Group, Employment Duration, and Annual Income. Based on the annual income entered by the user, the application automatically calculates the Monthly Income. These details help estimate the applicant's financial stability.

Step 7: Financial Risk and Credit Profile



The screenshot shows a form titled "3. Financial Risk & Credit Profile". It contains four input fields arranged in two rows. The first row has two input fields: "Requested Loan Amount (\$) *" with a dollar sign icon and a placeholder "e.g. 50000", and "Existing Debt Balance (\$) *" with a dollar sign icon and a placeholder "e.g. 15000 (0 if none)". The second row has two input fields: "Credit History Rating *" with a "Choose..." placeholder and a downward arrow, and "Credit Inquiries (Past 6 Months) *" with a placeholder "0". Below the form is a large dark blue button with a shield icon and the text "Evaluate Application Risk".

The final section collects financial information required for prediction. Users enter the Requested Loan Amount, Existing Debt Balance, Credit History Rating, and Number of Credit Inquiries. After completing all mandatory fields, the user clicks the **Evaluate Application Risk** button to submit the application for prediction.

Step 8: Prediction Result


APEX BANK

[Home](#)
[Prediction](#)
[Prediction History](#)
[Logout](#)

Vetting Transaction History

Review and audit recent credit card approval predictions processed in your current browser session.

Evaluation Logs



No Prediction Logs Found

You have not processed any credit applications in this session yet. Run predictions to populate this audit table.

[Start Evaluation](#)

Once the application is submitted, the Flask backend preprocesses the input data and sends it to the trained XGBoost model. The model predicts whether the application should be **Approved** or **Rejected**. The result page displays the prediction status, confidence score, prediction time, model used, and a detailed summary of all applicant information submitted for evaluation.

Step 9: Prediction History


APEX BANK

[Home](#)
[Prediction](#)
[Prediction History](#)
[Logout](#)

System Decision Report

Automated machine learning vetting completed on 2026-07-04 12:00:18



REJECTED

Confidence: 100.0%

Prediction Status

Application rejected due to too many recent credit inquiries, which signals high financial risk.

Model Used

XGBoost Classifier

Confidence

100.0%

Prediction Time

2026-07-04 12:00:18

[Check Another Applicant](#)

[View Prediction History](#)

Submitted Applicant Profile

Parameter	Submitted Value
Age	19.0
Annual Income	\$200,000.00
Applicant Name	Unnamed Applicant
Credit History	Good Credit
Credit Inquiries	20
Education Level	Incomplete higher
Employment Duration	0.0 years
Employment Status	Unemployed
Existing Loan Balance	\$0.00
Gender	Male
Housing Type	House / apartment
Income Type	Student
Loan Amount	\$3,000,000.00
Marital Status	Single / not married
Monthly Income	\$16,666.67
Number of Children	0
Occupation	Sales staff

Every prediction performed by the user is stored in the session and displayed in the Prediction History page. Users can review previous prediction results, compare applicant details, and track all evaluations performed during the current session. This feature provides better transparency and allows users to revisit previous predictions whenever needed.

Conclusion:

The **Credit Card Approval Prediction System** successfully demonstrates the application of Machine Learning in the banking and financial sector by providing fast, reliable, and data-driven credit approval decisions. The system is developed using the **Flask framework**, with an **XGBoost Classification Model** as the prediction engine, ensuring high prediction accuracy while maintaining a simple and user-friendly interface.

The application enables users to register, log in securely, submit applicant information through a structured web form, and receive an instant credit card approval prediction. Based on applicant demographics, employment details, income information, and credit history, the trained machine learning model evaluates the risk profile and classifies the application as **Approved** or **Rejected**. The prediction result is displayed along with the confidence score, applicant details, and model information, making the decision process transparent and easy to understand.

The project also includes additional features such as user authentication, automatic monthly income calculation, prediction history management, and responsive web pages, providing a complete end-to-end banking application experience. The integration of a modern frontend with a robust Flask backend demonstrates how machine learning models can be effectively deployed as real-world web applications.

Overall, this project improves the efficiency of the credit evaluation process by reducing manual effort, minimizing human errors, and delivering instant decision support. It also provides valuable practical experience in **Python programming, Machine Learning, Flask web development, data preprocessing, model deployment, and frontend-backend integration**. The developed system can serve as a foundation for future enhancements such as real-time bank database integration, explainable AI techniques, fraud detection modules, and cloud deployment, making it suitable for real-world financial decision support systems.